

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Shynbay Daniyal
Student Identification Number	230103158
Date & Laboratory Session	17.09.2026 02-N group
Workstation / Laptop Make & Model	(ASUS Vivobook 16 M1605YA)
Processor SKU / SoC Model	AMD Ryzen 5 5625U
Core Topology (Physical P-cores / E-cores)	Physical Cores: 6 Cores Logical SMT Threads: 12 Logical Processors
Cache Hierarchy Capacity	L1:32 KB/core (192 KB total) KB L2: 3.0 MB Total (512 KB per core) MB L3: 16.0 MB MB
Host OS & Python Runtime	Windows 11 Pro 64-bit Python Runtime: CPython 3.13.0

■ MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline T(1) and the measured multi-core execution times across all 6 core allocations.

```
import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")
```

"C:\Users\Daniyal Shynbay\AppData\Local\Programs\Python\Python313\python.exe" "C:\Users\Daniyal Shynbay\Desktop\SDU 7 Semester\Parallel Computing\3 Week\codes\task1_amdahl.py"

OS Reported Logical Cores: 12
Single Process Baseline Time T(1): 2.3256s

Cores	Time(s)	Observed Speedup	Theoretical Linear
1	2.2985	1.01x	1.00x
2	1.2369	1.88x	2.00x
4	0.7354	3.16x	4.00x
8	0.6327	3.68x	8.00x
12	0.5930	3.92x	12.00x
16	0.7308	3.18x	16.00x

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p×100]
p = 1 (Baseline)	2.2985	1.01x	1.00x	101.0%
p = 2	1.2369	1.88x	2.00x	94.0%
p = 4	0.7354	3.16x	4.00x	79.0%
p = 8	0.6327	3.68x	8.00x	46.0%
p = 12	0.5930	3.92x	12.00x	32.7%
p = 16	0.7308	3.18x	16.00x	19.9%

[Q1.1] Identify the exact inflection point (core count p*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

The speedup curve reaches its peak at **p = 12** cores, with a maximum observed speedup of **3.92×**. After that, performance decreases to **3.18×** at **p = 16**. The system has **12 logical cores**, corresponding to a **6-core / 12-thread CPU with SMT**. The inflection point occurs at **p = 12**, because adding more workers beyond the available logical threads introduces significant OS context-switching overhead, core contention, and scheduling delays. As a result, the additional workers do not provide further performance gains and instead reduce overall performance.

[Q1.2] Using your empirical measurements for T(1) and T(4), calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Student Response & Architectural Justification:

Using the baseline $T(1) = 2.2985$ s and $T(4) = 0.7354$ s, the observed speedup is $S(4) = 3.13\times$ (or $3.16\times$ using the table values).

Applying Amdahl's Law:

$$S(4) = 1 / [(1 - P) + P/4]$$

Therefore:

$$3.16 = 1 / (1 - 0.75P)$$

Solving for P gives:

$$P \approx 0.9113$$

This means that approximately **91.13% of the workload can be parallelized**, while the remaining **8.87%** is inherently sequential.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

Student Response & Architectural Justification:

Using the computed parallel fraction $P = 0.9113$, Amdahl's Law gives the maximum theoretical speedup:

$$S_{\max} = 1 / (1 - 0.9113) \approx 11.27\times$$

Therefore, even with a **128-core server node**, the speedup for this specific workload cannot exceed approximately **11.27 $\times$** due to the persistent serial portion of the program ($1 - P \approx 8.87\%$). This demonstrates that hardware scaling alone cannot overcome the fundamental algorithmic limitations of the workload.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache

and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ($16 * 8 \text{ bytes} = 128 \text{ bytes}$, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```
import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")
```

```
Running Adjacent (False Sharing)...
Running Padded (Cache-Aligned)...
Adjacent Trials: [0.908695600000101, 0.8959440999997241,
0.9109432000000197] | Median: 0.9087s
Padded Trials: [0.9176568999996562, 0.89870390000003278,
0.9022220000001653] | Median: 0.9022s
Slowdown Factor: 1.01x
```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.9087	0.8959	0.9109	0.9087	[Baseline]
Padded Indices (Cache-Line Aligned)	0.9177	0.8987	0.9022	0.9022	1.01x

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

Student Response & Architectural Justification:

In CPython, a primitive list element or reference occupies 8 bytes. With a standard L1 cache line size of 64 bytes, a single cache line can hold 8 elements ($64 / 8$). An index stride of 16 elements corresponds to $16 \times 8 = 128$ bytes. Since 128 bytes is greater than 64 bytes, adjacent threads access memory addresses located in separate, non-overlapping cache lines.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification:

When multiple threads write to adjacent indices on the same 64-byte cache line, Core 0 updates its index and changes the cache line state to **Modified (M)**. An invalidation signal is then sent to other cores. Core 1 receives this signal, forcing its local copy of the cache line from **Shared (S)** to **Invalid (I)**. When Core 1 attempts to write to the same cache line, it experiences a cache miss and must obtain the updated line from another core or memory, causing additional latency and pipeline stalls.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Student Response & Architectural Justification:

C++ alignas(64): Forces variables or struct members to be aligned to 64-byte boundaries, helping prevent multiple frequently accessed variables from sharing the same cache line.

Java @Contended Annotation: Adds internal padding around fields to reduce or prevent false sharing between frequently accessed variables stored on the same cache line.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

```

Unsafe: Value = 2,000,000 / 2,000,000 | Time: 0.1182s
Locked: Value = 2,000,000 / 2,000,000 | Time: 0.3799s
Lockless: Value = 2,000,000 / 2,000,000 | Time: 0.0602s
Contention Cost Multiplier (Locked/Unsafe): 3.21x
Lockless Speedup over Locked: 6.32x

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead

Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0 (<i>short burst</i>)	0.1182	0.31x
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.3799	[Baseline]

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

Under heavy thread concurrency, unprotected shared counters can suffer from lost updates because the increment operation is non-atomic. It consists of three separate steps: **LOAD** (read the value from memory into a register), **ADD** (increment the register by 1), and **STORE** (write the updated value back to memory). When multiple threads execute these steps simultaneously, their operations can overlap, causing one thread to overwrite another thread's update.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

[Q3.2 & Q3.3] The lockless thread-local accumulator completed in **0.0602 s**, achieving a **6.32× speedup** compared with the locked version, which took **0.3799 s**. Eliminating shared mutable state through thread-local buffers, following the **Map-Reduce pattern**, is more efficient because it removes the need for mutex acquisition, OS kernel context switches such as **futex** calls, and cache-line bouncing. This allows each thread to perform most of its work using its own local data, reducing synchronization overhead and improving overall performance.

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

[Q3.2 & Q3.3] The lockless thread-local accumulator completed in **0.0602 s**, achieving a **6.32× speedup** compared with the locked version, which took **0.3799 s**. Eliminating shared mutable state through thread-local buffers, following the **Map-Reduce pattern**, is more efficient because it removes the need for mutex acquisition, OS kernel context switches such as **futex** calls, and cache-line bouncing. This allows each thread to perform most of its work using its own local data, reducing synchronization overhead and improving overall performance.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```

import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()

```

```

=== Compute-Bound Suite (Register Math) ===
Workers: 1 | Time: 1.3034s | Scaling: 1.00x
Workers: 2 | Time: 1.6585s | Scaling: 1.27x
Workers: 4 | Time: 1.4296s | Scaling: 1.10x

=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===
Workers: 1 | Time: 0.5402s | Degradation: 1.00x
Workers: 2 | Time: 0.6248s | Degradation: 1.16x
Workers: 4 | Time: 1.0492s | Degradation: 1.94x

```

Worker	Compute-Bound	Compute	Memory-Boun	Memory

Allocation	Time [s]	Scaling [Tw/T1]	d Time [s]	Degradation [Tw/T1]
w = 1 Worker (Baseline)	1.3034	1.00x	0.5402	1.00x
w = 2 Workers	1.6585	1.27x	0.6248	1.16x
w = 4 Workers	1.4296	1.10x	1.0492	1.94x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

The compute-bound workload remains relatively stable across different numbers of workers, with execution time ranging from approximately **1.30 s to 1.43 s**. This is because most instructions execute within CPU registers and the L1/L2 caches without heavily stressing external memory resources. In contrast, the memory-bound workload increases from **0.5402 s** with 1 worker to **1.0492 s** with 4 workers, representing a **1.94× increase in execution time** due to memory bandwidth saturation.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

[Q4.2 & Q4.3] Consumer DRAM has a limited memory bandwidth, typically around **50–80 GB/s**. When multiple threads simultaneously stream large arrays, they can saturate the available memory channels. For a memory-bandwidth-saturated ML pipeline, optimizing the workload with **CUDA GPU kernels** can provide greater performance potential than simply increasing CPU thread parallelism, because GPUs can use high-bandwidth memory technologies such as **HBM or GDDR**, which can provide substantially higher memory bandwidth.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is

memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:
[Q4.2 & Q4.3] Consumer DRAM has a limited memory bandwidth, typically around **50–80 GB/s**. When multiple threads simultaneously stream large arrays, they can saturate the available memory channels. For a memory-bandwidth-saturated ML pipeline, optimizing the workload with **CUDA GPU kernels** can provide greater performance potential than simply increasing CPU thread parallelism, because GPUs can use high-bandwidth memory technologies such as **HBM or GDDR**, which can provide substantially higher memory bandwidth.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
[] Task 3 Unsafe Corruption Verified [] Task 2 Cache Slowdown Verified [] Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

--	--	--

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

count += i * i

return count

Run both experiments with total_work = 50_000_000 split across T

workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- The Empirical Curve:** Plot $T(n)$ (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- The Architectural Autopsy:**

- Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
- Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*? ◦ Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.